IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 20 March 2026, accepted 9 May 2026, date of publication 22 May 2026, date of current version 28 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3696039

RESEARCH ARTICLE

ACT-JEPA: Novel Joint-Embedding Predictive Architecture for Efficient Policy Representation Learning

ALEKSANDAR VUJINOVIĆ AND ALEKSANDAR KOVAČEVIĆ
Faculty of Technical Sciences, University of Novi Sad, 21000 Novi Sad, Serbia

Corresponding author: Aleksandar Kovačević (kocha78@uns.ac.rs)

This work was supported in part by the Ministry of Science, Technological Development and Innovation under Contract 451-03-34/2026-03/200156; and in part by the Faculty of Technical Sciences, University of Novi Sad through Project "Scientific and Artistic Research Work of Researchers in Teaching and Associate Positions at the Faculty of Technical Sciences, University of Novi Sad 2026" under Grant 01-3609/1.

ABSTRACT Learning efficient representations for decision-making policies is a challenge in imitation learning (IL). Current IL methods require expert demonstrations, which are expensive to collect. Additionally, they are not explicitly trained to understand the environment. Consequently, they have underdeveloped world models. Self-supervised learning (SSL) offers an alternative, as it can learn a world model from diverse, unlabeled data. However, most SSL methods are inefficient because they operate in raw input space. In this work, we propose ACT-JEPA, a novel architecture that unifies IL and SSL to enhance policy representations. It is trained end-to-end to jointly predict 1) action sequences and 2) latent observation sequences. To learn in latent space, we utilize Joint-Embedding Predictive Architecture, which allows the model to filter out irrelevant details and learn a robust world model. We evaluate ACT-JEPA in different environments and across multiple tasks. Our results show that it outperforms the strongest baseline in all environments. ACT-JEPA achieves up to 40% improvement in world model understanding and up to 10% higher task success rate. Finally, we show that predicting latent observation sequences effectively generalizes to predicting action sequences. This work demonstrates how integrating IL and SSL leads to efficient policy representation learning, an improved world model, and a higher task success rate.

INDEX TERMS Imitation learning, self-supervised learning, policy representation learning, latent space, world model.

I. INTRODUCTION

Learning end-to-end policies for decision-making tasks has long been a challenge in artificial intelligence. Recent advances in imitation learning (IL) have shown strong performance, with models learning from expert demonstrations [1], [2], [3], [4]. While these models can learn to mimic expert actions, they do not explicitly learn a world model — how the environment evolves over time. This limits their ability to predict future states and adapt to new situations [5], [6], [7], [8]. Another key issue with current IL methods is their reliance on high-quality expert data, which is both costly and time-consuming to collect [4], [7], [9]. Learning only from expert demonstrations limits the model's ability to generalize across scenarios, including failure cases, reducing its ability to recover from mistakes or handle novel situations [7], [10].

Self-supervised learning (SSL) provides an alternative, as it does not require labeled data and allows a model to learn from a broader range of experiences, even failure cases. However, most SSL methods make predictions in raw input space (e.g., pixel space), forcing the model to learn irrelevant or unpredictable details, which is computationally inefficient and requires large datasets [11], [12], [13], [14], [15]. This makes SSL less practical for real-world applications, especially in complex domains such as robotics.

The associate editor coordinating the review of this manuscript and approving it for publication was Syed Islam.

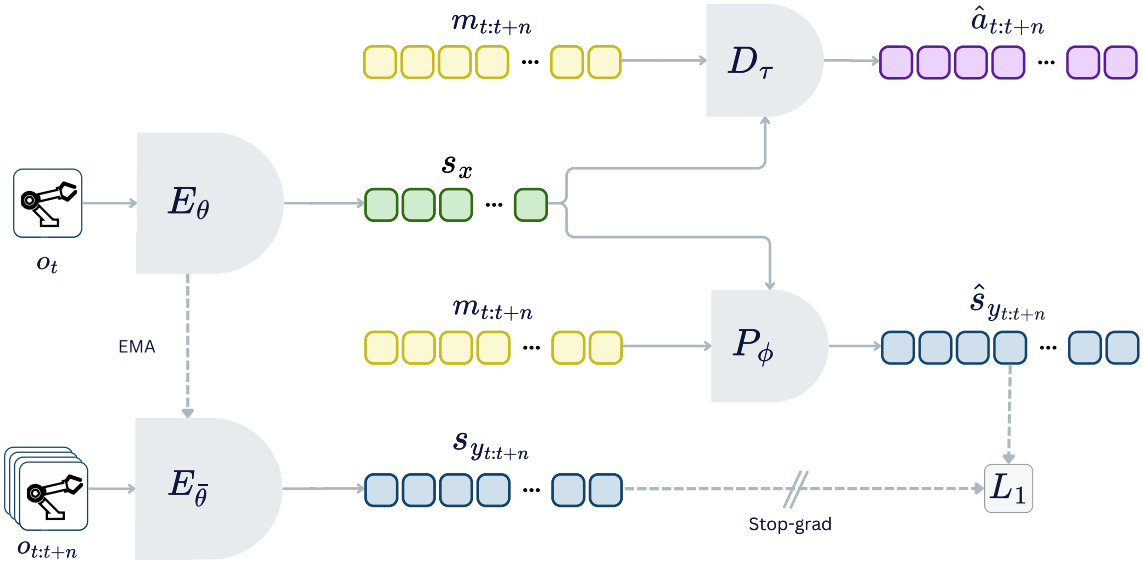


FIGURE 1. ACT-JEPA overview. The model is trained end-to-end to predict action sequences and latent observation sequences, with both objectives optimized using L_1 loss. It consists of four transformer-based components. The context encoder E_θ encodes the current observation o_t into a latent representation s_x. The target encoder $E_{\bar{\theta}}$ encodes future observations $o_{t:t+n}$ into target latent representations $s_{y_{t:t+n}}$. Then, the predictor P_ϕ takes s_x and learnable mask tokens to predict target latent representations $\hat{s}_{y_{t:t+n}}$. Finally, the action decoder D_τ takes s_x and learnable mask tokens to generate a sequence of actions $\hat{a}_{t:t+n}$. During inference, only E_θ and D_τ are utilized.

To address these limitations, we propose ACT-JEPA, a novel architecture designed to enhance policy representation learning. ACT-JEPA unifies IL and SSL objectives to predict both action sequences and latent observation sequences. It is trained end-to-end, jointly optimizing both objectives. The model learns to generate executable actions using IL, while simultaneously learning a world model using the SSL objective. To learn a world model in latent space, we utilize Joint-Embedding Predictive Architecture (JEPA) [11]. In contrast to SSL approaches that learn in raw input space, JEPAs learn by predicting in latent space, allowing the model to eliminate irrelevant and unpredictable information [11], [12], [13], [14], [15]. This approach makes learning efficient, develops a robust world model, and improves representation quality.

IL methods, such as behavior cloning, face challenges like compounding errors due to their autoregressive nature [1], [2], [4]. To resolve this, IL policies are often trained to predict action sequences instead of only one future action [4], [16], which is also utilized in ACT-JEPA. Building on this concept, we train ACT-JEPA to predict entire sequences of latent observations, instead of predicting only one future state [17], [18]. Expanding the prediction horizon results in a world model with richer and more robust representations. At the same time, learning in latent space makes the approach highly efficient.

We test our approach by running experiments across multiple tasks in simulated environments such as Push-T [3], Meta-World [19], and ManiSkill [20]. Our results show that ACT-JEPA outperforms other IL methods in all environments. We first show that ACT-JEPA has up to a 40% improvement in world model understanding compared to the strongest baseline. Similarly, we test ACT-JEPA's performance in the environments and show that it has up to a 10% higher task success rate. Finally, we show that learning latent observation sequences successfully transfers to predicting action sequences. These results show that ACT-JEPA is a promising direction for developing efficient, generalizable, and robust policy representations.

Learning robust policies with improved representations is particularly relevant in robotics, where sensor complexity and data volume are rapidly increasing. Modern robotic systems (e.g., robodogs, humanoids, and dexterous hands) integrate high-dimensional inputs from joints, cameras, and tactile sensors [21], [22], [23]. These systems generate large amounts of data, but not all information is equally important; some is unpredictable or irrelevant for a given task. By learning in latent space, ACT-JEPA can eliminate noise and focus on the most relevant features. By utilizing the SSL objective, the model can reduce reliance on expert data and learn from diverse data, including failure cases. Since the model is developed in latent space, it can naturally support various input modalities, making it well-suited for increasingly complex robotic platforms.

To summarize, our key contributions are:

- We present ACT-JEPA, a novel end-to-end architecture that learns to generate actions using imitation learning, while simultaneously learning a world model in latent space using JEPA. To the best of our knowledge, we are the first to combine IL and JEPA.
- By learning a latent world model, ACT-JEPA achieves up to 40% relative improvement over IL baselines in world model understanding.

- Across multiple decision-making benchmarks, ACT-JEPA consistently outperforms established IL baselines, achieving up to 10% absolute improvement in task success rate.
- Our joint optimization of IL and SSL world modeling outperforms both conventional two-stage training and IL-only policies under constrained data and computational budgets. This is particularly important in real-world robotic deployments where policies are developed for specific tasks, rather than general-purpose settings.

The rest of the paper is organized as follows. In Section II, we review related work. We present the ACT-JEPA architecture in Section III. Section IV explains the experimental setup to test our architecture, presents results, and discusses our findings. Section V concludes our paper. The training code and models are open source and available at `https://act-jepa.github.io`

II. RELATED WORK

A. IMITATION LEARNING

In imitation learning (IL), the agent learns from experts by imitating their behavior. The most common IL method is behavior cloning (BC), which formulates policy learning as a supervised learning task: the agent is trained to predict the expert's actions using a dataset of state-action pairs. Many recent works have tried to improve policies by focusing on different architectures and objectives. For example, some utilized large pretrained models and adapted them for downstream decision-making tasks [1], [2], [24], [25]. These models are autoregressive, trained to predict the next action token based on inputs like images and text instructions. While they show promise, they are constrained by their autoregressive nature to predict the next token. This leads to several limitations. First, prediction errors accumulate, which can push the model into states outside the training distribution [4]. These models struggle with non-Markovian behavior, like pauses during demonstrations (e.g., when a human demonstrator stops to think or plan their next actions) [4]. Continuous actions are discretized into predefined bins (tokens), requiring the selection of the optimal size and number of bins [25], [26]. These models utilize a finite token vocabulary, which limits their ability to handle action spaces with a large or infinite number of actions. Finally, large models require remote hosting and cannot run on resource-constrained devices like drones or self-driving cars [2].

Other works have explored non-autoregressive generative models. For example, some works utilized autoencoders [4], [16], others used diffusion models [3], [27], [28], and recent works have explored flow matching models (closely related to diffusion models) [10]. These models are well-suited for continuous action data, eliminate the need for discretization, can handle non-Markovian problems, and help mitigate compounding errors. The key is to predict an action sequence (i.e., action chunk) instead of a single action. Despite the progress, these methods heavily rely on labeled expert action data for training, which is costly and time-consuming [4], [7]. Utilizing only expert data limits their ability to generalize to unseen scenarios or recover from failures, unless explicitly demonstrated during training [7], [10].

1) ACT

Action Chunking with Transformer (ACT) [4] is a non-autoregressive transformer-based policy trained to predict action sequences from observations. It can be implemented as a variational autoencoder (VAE) or a simpler autoencoder (AE). By training the model to minimize the L_1 loss on continuous action sequences, it avoids discretization and helps mitigate compounding errors and non-Markovian data. Among IL approaches, this one is the closest to our method, as both methods predict action chunks with a transformer-based architecture. However, ACT-JEPA additionally develops a world model and enables a deeper understanding of the environment from unlabeled data. The specific ACT variant used in our experiments is described in subsection IV-B.

B. REPRESENTATION LEARNING WITH JOINT-EMBEDDING PREDICTIVE ARCHITECTURES

Self-supervised learning (SSL) has revolutionized representation learning in domains such as natural language processing and computer vision. Through SSL, models can learn rich representations from a range of unlabeled data. Various objectives and transformer-based architectures have been explored, such as GPTs [29], [30], autoencoders [31], [32], and diffusion models [33], [34]. Their key drawback is a reconstructive objective: the model must predict complete representations, including irrelevant or unpredictable details [12], [14], [15], [17]. Consequently, these architectures are computationally expensive and require large training datasets, which results in lower quality learned representations. Joint Embedding Predictive Architectures (JEPAs) overcome these limitations by learning to predict in latent space, where the model is free to discard irrelevant and unpredictable details. This improves both representation learning and computational efficiency.

The first concrete JEPA implementation is the I-JEPA approach [12]. The main idea is to mask parts of the input and predict the masked parts in latent space using a simple reconstruction loss in embedding space. For example, I-JEPA learns to predict masked blocks of images from the visible regions. Importantly, this is done without hand-crafted augmentations (e.g., scaling or rotation), which reduces inductive bias and provides a more general solution that can be applied beyond images. In video representation learning, the model learns to predict masked temporal tubes to capture spatial and temporal dynamics [13]. In the audio domain, it has been used to extract meaningful features from spectrograms [35]. In 3D data processing, it effectively learns representations from point-cloud data [36]. It has also proven effective for learning touch representations in tasks such as slip detection and grasp stability [37].

This broad applicability highlights JEPA's potential to drive improvements across a range of tasks and modalities.

C. SELF-SUPERVISED LEARNING FOR POLICY REPRESENTATION

In the context of policy learning, most SSL methods are focused on improving policy representation by learning good visual representations in pixel space [38], [39], [40], [41]. Only recently have some SSL methods shifted focus towards learning environment dynamics in latent space.

1) DynaMo

This approach employs a two-stage training process to first learn environment dynamics and subsequently learn a policy to output actions [17]. In the first stage, an encoder is pretrained on in-domain task images to learn environment dynamics (both inverse and forward dynamics models). To make this efficient, the environment dynamics are learned in latent space using the VICReg objective [42]. In the second stage, the frozen pretrained encoder is leveraged to train an action decoder head with IL. Experiments showed that DynaMo's abstract representations are more robust than those learned by SSL methods operating in input space, outperforming the baselines in decision-making.

2) DINO-WM

This method is trained to develop a world model from images in latent space, rather than pixel space [18]. It captures environment dynamics between two successive frames. Specifically, it predicts the latent representation of the next frame z_{t+1}, given the latent representation of the current frame z_t, and the current action a_t. The model is optimized in embedding space with L_2 loss. To encode frames and obtain latent representations, DINO-WM uses pretrained DINO-v2, which achieves the best performance but is significantly larger than common backbones like ResNet-18. Instead of training an action decoder with IL, DINO-WM uses model predictive control (MPC), a common algorithm used in planning and control. Although this approach results in a robust world model, its pretraining requires access to ground-truth actions.

3) V-JEPA 2-AC

Most recently, V-JEPA 2-AC was released after the initial publication of ACT-JEPA [43]. It employs a two-stage approach to learn a general-purpose world model: first, JEPA is used to pretrain an encoder on general-purpose video datasets; then, the pretrained encoder is frozen, and a separate latent predictor model is trained on robotics data with ground-truth actions to learn environment dynamics. Like DINO-WM, it uses MPC to sample actions.

While these studies show promise, they leave room for improvement. Their experiments primarily focus on comparing policy performance against SSL methods operating in pixel space. Moreover, their methods differ from ours in objectives, architectures, and overall goals. We summarize the main differences: DynaMo adopts a two-stage process with separate pretraining and policy learning, whereas our method is trained end-to-end. This is generally more scalable and effective, as the learned representations are jointly optimized for downstream tasks; our experimental results also support this claim. Additionally, DynaMo relies on more complex pretraining procedures and objectives compared to our approach. In contrast to DINO-WM and V-JEPA 2-AC, which rely on MPC for action selection, our approach directly learns actions through IL. Both MPC and IL have their respective strengths. However, IL can effectively operate in noisy real-world environments, whereas MPC performance is limited by the quality of a learned world model. Additionally, MPC is usually slower as it requires optimization at inference time. Finally, in contrast to these methods, ACT-JEPA predicts entire sequences of actions and latent observations, which mitigates compounding errors and develops richer representations.

While SSL has shown promise in representation learning across various domains, its application to policy learning remains underexplored. These challenges highlight the need for approaches that can efficiently improve internal representations, learn a world model, and enhance decision-making. ACT-JEPA addresses these gaps by combining IL and SSL to learn action sequences and latent observation sequences end-to-end.

III. ACT-JEPA

In this section, we describe ACT-JEPA, illustrated in Figure 1. It unifies IL and SSL objectives to learn executable actions and develop a world model. We train it end-to-end, jointly optimizing both objectives. To learn executable actions, the model is trained to predict a sequence of n future actions using IL. Predicting action sequences, instead of a single future action, improves performance and mitigates issues such as compounding errors and non-Markovian behavior in the data [4]. To develop a world model, the model is trained to predict latent observation sequences using SSL. Predicting entire sequences, instead of a single future state, improves world model understanding. At the same time, learning in latent space makes this approach highly efficient; to achieve this, we build on the recent JEPA architecture. Notably, SSL is used to learn how the environment evolves, not to imitate actions; for example, even if a non-expert demonstration shows a car driving off a cliff, the model does not learn to perform those actions. Instead, it learns the states leading to that scenario, improving its world model. In the following, we describe the theoretical background, the main components of our architecture, and the objectives.

A. BACKGROUND

In this section, we provide the theoretical foundation for the ACT-JEPA architecture. We introduce key concepts in robot learning, such as imitation learning and behavior cloning. Then, we describe JEPA as a novel self-supervised approach.

1) POLICY LEARNING

Imitation learning methods train a policy π on a dataset of expert demonstrations $\mathcal{D}$ to mimic the expert's behavior. Each demonstration (episode) represents the full interaction from the start step $t = 0$ to the end (termination) step $t = T$. A demonstration consists of actions and observations. All episodes in the dataset are successful, meaning that the given task was successfully solved by the expert. Behavior cloning is a common algorithm that casts imitation learning as supervised learning, mapping observations to actions. The policy π produces action(s) given some observation(s) to minimize the difference between the predicted actions and the expert's actions. In general, the policy can be defined as $\pi\ (a_{t:t+n} \mid o_t)$, producing a sequence of n future actions. At test time (inference), the policy operates iteratively: every n steps, it receives observation(s) o_t, generates a sequence of n future actions $a_{t:t+n}$, and executes them.

2) JOINT-EMBEDDING PREDICTIVE ARCHITECTURE

Self-supervised learning (SSL) is a representation learning method in which the system learns useful representations from its inputs. The Joint-Embedding Predictive Architecture (JEPA) is a novel approach within SSL that learns to produce similar embeddings for compatible inputs x and y, and dissimilar embeddings for incompatible inputs [12]. The loss function is applied between embeddings in latent (abstract) representation space, not raw input space (e.g., pixel space). Consequently, this elevates the level of abstraction and brings benefits such as improved training time and generalization.

In JEPAs, the model architecture consists of three main components: context encoder, target encoder, and predictor, as in Figure 1. The context encoder f_θ takes an input x and produces a context representation s_x. The target encoder $f_{\overline{\theta}}$ encodes a target y and produces a target representation s_y. To predict the target representation s_y, the predictor g_ϕ takes in the output of the context encoder s_x and a (possibly latent) variable z, and outputs the predicted target representation $\hat{s}_y$. For example, in the I-JEPA implementation [12], the context encoder was utilized to process one segment of an image, while the target encoder processed another segment (e.g., a missing segment of an image). The predictor then used the context representation to predict the representation of the missing segment. The loss function is usually implemented as the L_1 or L_2 distance between the target representation s_y and the predicted target representation $\hat{s}_y$ [12], [13]. To train a JEPA model, the parameters of the context encoder θ and the predictor ϕ are typically updated with gradient descent. The target encoder is often implemented as a copy of the context encoder and is updated via exponential moving average (EMA) of the context-encoder parameters.

B. ARCHITECTURE OVERVIEW

ACT-JEPA is an end-to-end architecture that efficiently learns representations relevant for decision-making and world model understanding (Figure 1). The architecture consists of four main components: context encoder, target encoder, predictor, and decoder. At the core of each component is a transformer architecture [44]. All components are utilized during training. During inference, only the context encoder and decoder are utilized to generate action sequences, discarding the target encoder and predictor.

The inputs to the model are observations of different modalities. In our implementation, we set the context encoder to receive a current image and the proprioceptive state, while the target encoder receives a sequence of proprioceptive states. We note that the architecture is more general and can support various modalities as inputs (e.g., images, depth images, proprioceptive states, pose markers, tactile feedback, images from different camera views).

1) CONTEXT ENCODER

The context encoder E_θ takes as input observations at a timestep t and outputs a latent representation of the environment s_x. The output s_x is shared between the tasks of predicting action sequences and latent observation sequences, which requires the context encoder to capture information relevant to both tasks. This design improves the model's internal representations, understanding of environment dynamics, and decision-making.

We implement the context encoder to receive an image, the proprioceptive state, and a task label. Each modality is encoded with a dedicated function to obtain modality-specific tokens. For instance, proprioceptive states are encoded using linear layers. Task labels are one-hot encoded. To process images, we follow the common approach of utilizing the pretrained ResNet-18 model to extract feature maps [4], [16], [17], [26]. The feature maps are flattened along the spatial dimension to form a sequence of tokens, with positional encoding applied to preserve spatial relationships. ResNet-18 is chosen for its simplicity and effectiveness. While our approach aligns with standard architectures, the flexibility of our architecture allows for the integration of more advanced backbones such as DINO-v2 [18], [45] or conditioning mechanisms like FiLM [2], [16], [46], which may yield improved performance in more complex tasks. Once all modalities are encoded into tokens, they are concatenated into a single sequence and fed into the transformer model. The model outputs the latent representation s_x, which serves as the input to the subsequent decoder and predictor components.

2) TARGET ENCODER

The target encoder receives an observation sequence $o_{t:t+n}$ as input and outputs a sequence of latent observations $s_{y_{t:t+n}}$. These representations capture how the environment evolves over time. They also serve as the targets for the second objective (predicting latent observation sequences). By representing targets in latent space, the model eliminates irrelevant or unpredictable details from the target representation.

We encode the input sequence with a modality-specific projection function. We select proprioceptive states as the target modality and encode them with a linear projection layer (as was done in the context encoder). Then, positional encoding is added to preserve temporal information. The encoded tokens are fed into the transformer model to process them. The output is a sequence of tokens $s_{y_t}, \ldots, s_{y_{t+n}}$, where each token is a latent observation. Thus, the whole output s_y represents the sequence of latent observations.

By outputting a sequence, the target encoder captures temporal environment dynamics. In contrast, the context encoder captures a representation of the environment at the current timestep t only. This enables the target encoder to produce non-trivial and semantically meaningful targets, while the context encoder provides informative context for downstream tasks. Under these settings, we align with prior works to design non-trivial prediction tasks while maintaining an informative context.

3) PREDICTOR

The predictor P_ϕ learns how the environment evolves over time. It receives two inputs: the output of the context encoder s_x and a sequence of n mask tokens $m_{t:t+n}$. Each mask token is a learnable vector and corresponds to an abstract observation we wish to predict. Instead of simply concatenating both inputs and passing them through a transformer model, we use a cross-attention block to condition on the context s_x. This enriches the mask tokens with contextual information before they are processed by a transformer model. This is in contrast to similar JEPA architectures that use self-attention over all inputs [12], [13], [17]. The cross-attention blocks have linear complexity with respect to context length, allowing our model to efficiently handle longer sequences [47]. By using cross-attention, we achieve a more flexible and efficient processing mechanism, as we reduce the number of tokens processed by the subsequent transformer blocks. The output of the predictor is a sequence of tokens $\hat{s}_{y_t}, \ldots, \hat{s}_{y_{t+n}}$, where each token represents the predicted latent observation. We select proprioceptive states as the observation modality, consistent with the modality used in the target encoder.

4) DECODER

We utilize the action decoder D_τ to predict action sequences. It takes in the output of the context encoder s_x and a sequence of n mask tokens $m_{t:t+n}$. Each mask token is a learnable vector and corresponds to an action we wish to predict. As in the predictor, we use a cross-attention block to condition on s_x, adding contextual information to the mask tokens, which are then processed by a transformer model. The output is a sequence of predicted actions to be executed in the environment $\hat{a}_t, \ldots, \hat{a}_{t+n}$.

The independent action decoder adds flexibility to the architecture. This is aligned with similar JEPA works that use independent decoder heads for downstream tasks (e.g., classifying images or generating pixels) [12], [13]. In our context of generating actions, it allows for easy substitution with more sophisticated generative models, such as diffusion [3], [16]. However, more complex action decoders are often unnecessary if the encoder outputs good representations [16].

C. OBJECTIVE

We train the model with two objectives: given a current observation o_t, predict 1) an action sequence $\hat{a}_t, \ldots, \hat{a}_{t+n}$ and 2) a latent observation sequence $\hat{s}_{y_t}, \ldots, \hat{s}_{y_{t+n}}$. The first objective is supervised, requiring expert actions, while the second objective is self-supervised, allowing the model to learn from unlabeled observation data. These objectives ensure the model learns both low-level actions and high-level environment dynamics. To optimize both objectives, we utilize two loss functions.

1) ACTION LOSS

For the first objective, we aim to reconstruct a sequence of future actions. Predictions are made in the given action space. To achieve this, we utilize L_1 loss, following [4]. The loss penalizes distances between the predicted and the target actions:

$$\mathcal{L}_{actions} = \frac{1}{n} \sum_{i=0}^{n} \left\| \hat{a}_{t+i} - a_{t+i} \right\|_1 . \tag{1}$$

2) OBSERVATION LOSS

For the second objective, L_1 is also utilized as it was the most efficient in our experiments. Here, predictions are made in latent space, rather than the given observation space. Therefore, the loss penalizes the distance between the predicted latent observations and their corresponding targets:

$$\mathcal{L}_{observations} = \frac{1}{n} \sum_{i=0}^{n} \left\| \hat{s}_{y_{t+i}} - s_{y_{t+i}} \right\|_1 . \tag{2}$$

3) FINAL LOSS

The model is trained end-to-end by jointly optimizing both objectives. The final loss is computed as the sum of the two individual losses:

$$\mathcal{L} = \mathcal{L}_{actions} + \mathcal{L}_{observations}. \tag{3}$$

This is in contrast to most approaches that separate SL and SSL into two distinct training stages [10], [12], [13], [37]. In the two-stage approach, the model is first pretrained with SSL to learn useful representations, and is then adapted to a downstream task using SL, often through a task-specific decoder (e.g., image classification). Most of the model's knowledge is acquired during the pretraining stage, but this requires large amounts of pretraining data and computational resources. In our experiments, we found that combining both objectives into a single training stage is crucial for achieving strong performance. Jointly optimizing both losses in an end-to-end manner proved more effective than the two-stage approach.

Following the prior JEPA works [12], [13], [35], the parameters of the context encoder, predictor, and decoder (θ, ϕ, τ) are learned through gradient-based optimization. The parameters of the target encoder $\overline{\theta}$ are updated with the exponential moving average of the context encoder parameters, which is necessary to prevent representation collapse in latent space.

IV. EXPERIMENTS

In this section, we evaluate ACT-JEPA's performance in world model understanding and decision-making. We first describe environments used for evaluation. Next, we outline the baselines used for comparison. Finally, we present experiments designed to answer the following research questions:

- Does ACT-JEPA improve world model understanding by learning latent observation sequences?
- Does learning a world model with JEPA capture features useful for policy learning?
- How does joint optimization of IL and SSL objectives compare to optimizing them separately?
- Does ACT-JEPA improve decision-making compared to established baselines?

A. ENVIRONMENTS

We describe the environments in which we test the hypotheses. All tasks in the environments have continuous observation and action spaces. Figure 2 shows representative observations from the benchmarks.

1) PUSH-T

This is a 2D environment in which an agent (circle) has to push a moveable T-shaped object to a fixed T-shaped target area [3]. The action space and proprioceptive space are two-dimensional. This task requires precise object manipulation as the task is considered solved when the T-shaped object covers the target area. The dataset consists of 206 human-collected demonstrations with RGB images of size (96, 96). Due to the low resolution, which makes fine-grained coverage difficult to assess reliably, we define the task as successful when the object covers at least 90% of the target area.

2) META-WORLD

This environment suite consists of a diverse set of tasks that share the same agent and workspace [19]. It combines motions (e.g., reach, push, grasp) with varying objects to create tasks such as opening a drawer, opening a door, or pushing a button. This setup ensures a shared underlying connection between the tasks, as they share similar environment dynamics. This connection is crucial as it allows for representations learned in one task (e.g., opening a drawer) to transfer effectively to another (e.g., opening a door). The robot itself has four joints, therefore the proprioceptive state and the action space are four-dimensional. We used RGB images of size (128, 128) to balance computational efficiency with sufficient resolution for extracting meaningful features, as this proved to be effective in our experiments. We selected 15 different tasks and utilized a scripted policy to collect the training data. To focus on generalization and shared environment dynamics, we selected tasks that rely on a common set of manipulation primitives (e.g., reaching, pushing, opening), while excluding tasks that do not share these core operations (e.g., kicking a ball into the goal). We collected 40 demonstrations per task, each with a different initial state (e.g., varying object positions).

3) ManiSkill

ManiSkill is a suite of simulated 3D robot manipulation tasks designed to evaluate object manipulation and generalization in diverse scenarios [20]. To ensure a consistent evaluation framework, we filtered the tasks by first identifying tasks with available expert demonstration datasets. We then narrowed this selection to those utilizing the same robot (a two-finger gripper arm) and the same action space. This resulted in five tasks (pick cube, push cube, stack cube, pull cube with a tool, and insert a peg sideways). We used 50 demonstrations per task with RGB images of size (128, 128).

B. BASELINES

We compare ACT-JEPA with two behavior cloning methods, representing well-established baselines. To provide a fair comparison with the proposed architecture, we chose baseline policies that have the following in common: (1) visual data is preprocessed by a CNN-based backbone, (2) the transformer architecture is at the core of each component, and (3) each policy model was designed to directly output actions in continuous space. In the following, we describe the baselines in detail.

1) AUTOREGRESSIVE (AR) TRANSFORMER

This is a GPT-style autoregressive (AR) transformer decoder policy. The input is a sequence of previous actions and observations (e.g., images) represented as tokens. The policy predicts the next action based on this sequence, leveraging the transformer's attention mechanism to effectively model long-term dependencies. It is trained to minimize the L_2 loss between the predicted and actual action. This baseline is equivalent to Decision Transformer (DT) [48], but without using rewards as input. It is also similar to Behavior Transformer (BeT) [25], but without action discretization. Both DT and BeT are trained from scratch and have been widely adopted in similar settings. Thus, we choose the AR transformer baseline, as it requires minimal modifications to the original GPT framework. This makes it a relevant and effective baseline.

2) ACT

Another baseline we utilize is Action Chunking with Transformer (ACT) [4]. We implement ACT and ACT-JEPA to match in terms of inputs, outputs, and the training

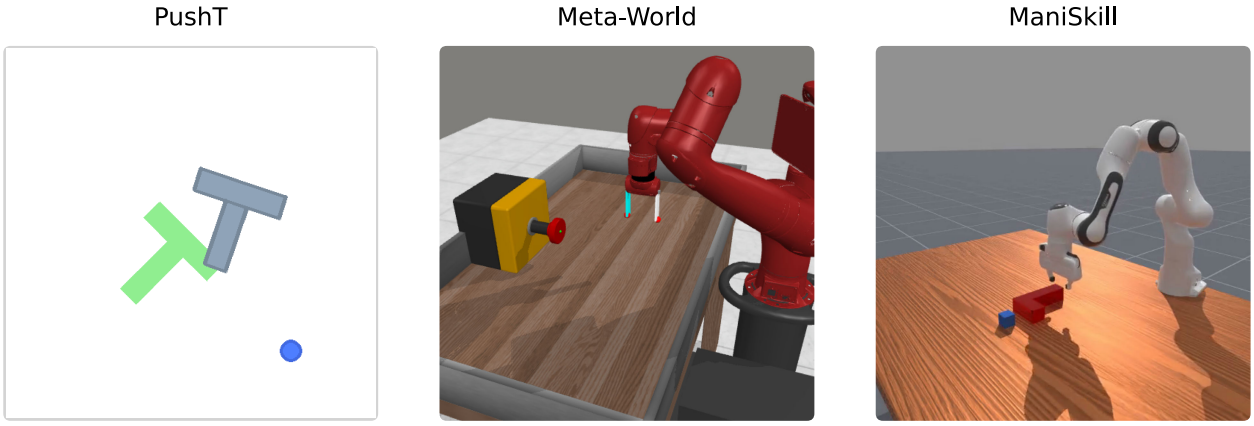


FIGURE 2. Benchmark environments used for evaluation. ACT-JEPA is evaluated on Push-T, Meta-World, and ManiSkill, covering a range of 2D and 3D robotic manipulation tasks.

objective, enabling a controlled comparison. We use the AE variant instead of VAE, removing the variational encoder component to simplify the architecture without sacrificing performance in our experiments. This architecture is similar to the recent BAKU architecture [16], which achieves state-of-the-art results in similar environments. However, we do not add BAKU's additional extensions, such as observation history, FiLM conditioning [46], or complex text-based conditioning. While it is possible to implement these modifications, we intentionally omit them to maintain the focus on learning a world model, not on incorporating additional features that may distract from this objective. Therefore, ACT is the most relevant baseline for our method. The key architectural difference is that ACT uses only IL, while ACT-JEPA additionally utilizes JEPA to learn a world model.

C. DOES ACT-JEPA IMPROVE WORLD MODEL UNDERSTANDING BY LEARNING LATENT OBSERVATION SEQUENCES?

Effective policy representations should capture information relevant for action prediction and world modeling. ACT-JEPA is designed to address this by learning both action sequences and latent observation sequences. To assess whether learning latent observation sequences improves world model understanding, we test if the learned representations can be used to predict future states. We evaluate representation quality using a probing task [13]. For each trained policy, we freeze the context encoder, discard other components, and train a randomly initialized decoder to reconstruct future states from the frozen representations. The observation modality is proprioceptive states, as this was the selected target modality in our experiments. To evaluate performance, we use Root Mean Squared Error (RMSE) and Absolute Trajectory Error (ATE) as metrics [47], [49]. RMSE highlights larger deviations by penalizing them more, giving an overall measure of error magnitude, while ATE measures how far off the predicted trajectory is from the ground truth trajectory. Together, these metrics offer a comprehensive evaluation of both the magnitude and alignment of the trajectory.

Our findings are presented in Table 1. ACT-JEPA consistently outperforms the ACT baseline across all benchmarks, reducing the prediction error by 29–37% (RMSE) and 29–40% (ATE). These consistent improvements across diverse tasks demonstrate that ACT-JEPA learns more efficient and accurate world model representations.

To understand how representations evolve during training, we repeat the experiment at regular intervals throughout training. This approach reveals not only the final performance, but also how representation quality improves over time.

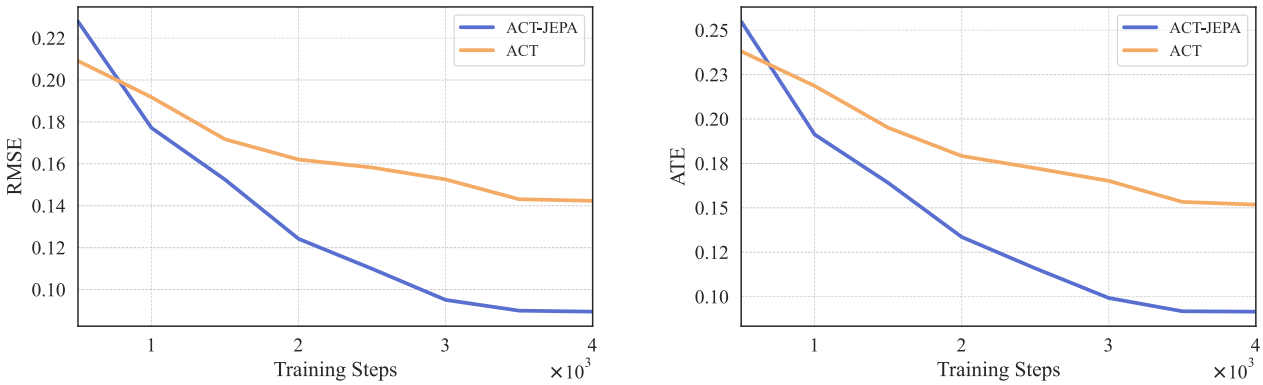
Results from the ManiSkill environment are shown in Figure 3; similar trends are observed in other environments. Although ACT-JEPA may start with a higher error in both metrics, it quickly overtakes the baseline. As training progresses, ACT-JEPA decreases prediction error faster, while ACT plateaus at higher error values. This indicates that ACT-JEPA converges faster while achieving a lower error in both RMSE and ATE. Predicting in latent space makes this approach efficient, aligning with recent JEPA works [12], [13].

D. DOES ACT-JEPA IMPROVE DECISION-MAKING COMPARED TO ESTABLISHED BASELINES?

We evaluate ACT-JEPA's performance on decision-making tasks and benchmark it against the baseline policies. Our primary metric is task success rate, defined as the percentage of successfully solved tasks in the environment. Each policy is a transformer-based model trained from scratch on the same number of tokens per environment. Policies are evaluated across diverse initial states per task to ensure robust evaluation. For example, in the Meta-World environment, we evaluate 20 distinct initial states per task across 15 tasks ($20 \times 15 = 300$ total evaluations). To isolate the effect of representation learning objectives and enable a fair comparison with ACT, ACT-JEPA uses the same action-decoder head as ACT. While more expressive decoders (e.g., diffusion-based) may improve performance in complex

TABLE 1. World model evaluation. The quality of learned representations for world modeling is evaluated using a probing task. We freeze the encoder weights and train a decoder head on its representations to output sequences of future proprioceptive states. ACT-JEPA consistently outperforms the ACT baseline across benchmarks, achieving 29–40% lower error, indicating that ACT-JEPA captures more information about environment dynamics. Lower values are better.

	Push-T		ManiSkill		Meta-World	
Method	RMSE ↓	ATE ↓	RMSE ↓	ATE ↓	RMSE ↓	ATE ↓
ACT	0.1424	0.1518	0.0531	0.2063	0.0295	0.0529
ACT-JEPA	**0.0895**	**0.0915**	**0.0348**	**0.1354**	**0.0208**	**0.0375**



(a) ManiSkill RMSE ↓ (b) ManiSkill ATE ↓

FIGURE 3. World model representation quality during training. RMSE (left) and ATE (right) for predicting future proprioceptive states throughout training in the ManiSkill environment. At fixed checkpoints, we freeze the context encoder and train a randomly initialized decoder to predict future states from the frozen representations, then evaluate its prediction error. Lower values indicate better predictive representations. ACT-JEPA reaches lower error earlier and maintains lower error overall, demonstrating more effective learning of environment dynamics.

environments, our model naturally supports such alternatives, which we leave to future work.

We report the average task success rate results in Table 2. ACT-JEPA outperforms all baselines across the selected environments. Compared to the AR transformer policy, ACT-JEPA achieves an absolute improvement of +41% in Push-T, +28% in ManiSkill, and +53.7% in Meta-World. We attribute the AR policy's lower performance to compounding errors and non-Markovian behavior in the data, consistent with [4]. In contrast, action chunking in ACT and ACT-JEPA mitigates these issues and improves performance.

Compared to the ACT policy, ACT-JEPA improves success rate by +7% in Push-T, +10% in ManiSkill, and +2% in Meta-World. The smaller improvement in Meta-World can be attributed to ACT already achieving near-ceiling performance (90% success rate), leaving limited room for further gains. We observe larger relative improvements of ACT-JEPA in environments where performance is not saturated, suggesting that learning a JEPA-based world model in addition to IL enhances policy robustness and generalization. Overall, these results demonstrate that ACT-JEPA consistently improves decision-making performance across environments.

E. DOES LEARNING A WORLD MODEL WITH JEPA CAPTURE FEATURES USEFUL FOR POLICY LEARNING?

We isolate JEPA components and test if a learned world model captures policy-relevant features. Specifically, we test if learning latent observation sequences transfers effectively to downstream tasks, such as predicting actions. The assumption is that representations learned through one task (predicting observation sequences) can transfer to another (predicting action sequences), if there is an underlying connection between them.

To evaluate this, we design a two-stage training process: the model is first pretrained with SSL to learn latent observation sequences, after which the encoder is frozen and a newly initialized action decoder is trained with IL to predict action sequences. In the pretraining stage, the goal is to learn useful representations that effectively transfer to downstream tasks, such as action prediction. We utilize only JEPA components (context encoder, predictor, and target encoder) and discard the action decoder. The model is pretrained to minimize the L_1 loss between predicted and target latent observation sequences (Equation 2). In the second stage, we append a randomly initialized action decoder to the frozen encoder (as in Figure 1) and train it to reconstruct action sequences using the IL objective (Equation 1). To track representation quality over time, we periodically probe the encoder during pretraining. Every n steps, we freeze the encoder, train a new decoder from scratch, and record its action reconstruction loss and task success rate. The decoder is then discarded, and pretraining resumes with the original encoder.

As shown in Figure 4, the action reconstruction loss consistently decreases throughout training. This indicates that

TABLE 2. Policy performance. Average task success rate (%) across the evaluated benchmarks. Results show that ACT-JEPA consistently improves performance over imitation learning baselines across all environments.

Method	Push-T (1 task) ↑	ManiSkill (5 tasks) ↑	Meta-World (15 tasks) ↑
AR transformer	0%	8%	38.3%
ACT	34%	26%	90%
ACT-JEPA	**41%**	**36%**	**92%**

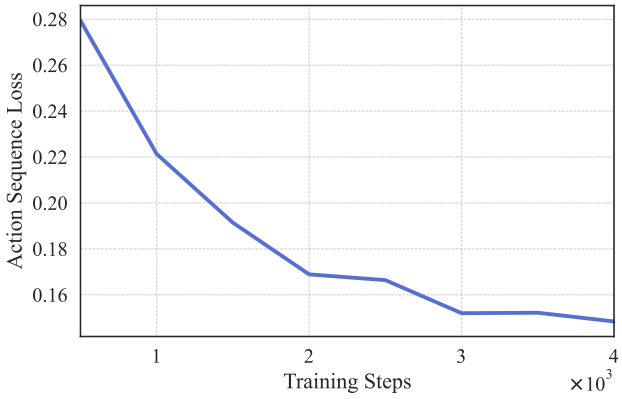


FIGURE 4. Action prediction probe during JEPA pretraining. During SSL pretraining with JEPA, we periodically freeze the encoder, attach a newly initialized action decoder, and train the decoder to predict future action sequences. The figure shows results for the Push-T environment, while similar trends are observed in others. As pretraining progresses, the probing action decoder achieves lower reconstruction loss, showing that the learned latent dynamics become increasingly useful for downstream policy learning.

the model gradually refines its internal representations during pretraining. Although limited by dataset size, pretraining shows improvement in representation quality for downstream tasks. This suggests that the learned world model could be utilized for different control tasks, including planning and alternative prediction objectives. Importantly, the experiments confirm that learning a JEPA world model captures features useful for policy learning.

F. HOW DOES JOINT OPTIMIZATION OF IL AND SSL OBJECTIVES COMPARE TO OPTIMIZING THEM SEPARATELY?

Combining SSL and SL is often implemented in a two-stage process: the model is first pretrained with SSL and then adapted to a downstream task with SL [12], [37], [45]. This is also common in policy learning, where pretraining is followed by IL to learn actions [10], [17], [40]. In subsection IV-E we demonstrated that such a two-stage approach is effective: pretraining captures representations useful for predicting both future states and actions. However, it remains unclear whether optimizing both objectives separately is the most effective approach. To investigate this, we compare the two-stage approach (subsection IV-E) with an end-to-end approach that jointly optimizes SSL and IL objectives (Equation 3).

As shown in Table 3, joint optimization outperforms the two-stage approach on the reported benchmarks. We attribute this performance gap to a representation misalignment in the two-stage approach. During SSL pretraining, the model primarily focuses on modeling environment dynamics. Although these representations are informative, they lack information required for fine-grained action generation. Consequently, training only the action decoder on top of frozen representations cannot fully compensate for this mismatch, leading to worse performance than the end-to-end model. This limitation is critical in robotics, where even small action errors can accumulate over time and cause task failure.

In contrast, joint optimization encourages the model to learn shared representations, allowing gradients from both objectives to shape the representation space. By aligning both objectives throughout training, the representations become more coherent, robust, and generalizable, resulting in a more effective policy. Joint optimization also provides a mutual regularization effect: the JEPA objective prevents overfitting to demonstration data by modeling environment dynamics, while the IL objective ensures the world model prioritizes control-relevant features for action generation. It also simplifies the training pipeline by eliminating the need for multi-stage scheduling and managing fine-tuning phases.

These findings are particularly relevant for robotic systems developed for specific tasks under constrained data and computational resources. In such settings, collecting demonstrations is costly and time-consuming, and deploying large pretrained models on robotic platforms may not be feasible. From a practical perspective, replacing multi-stage scheduling with end-to-end training simplifies engineering pipelines. How these findings extend to large-scale pretraining remains an open question. The two-stage approach may be advantageous when representations are reused across distinct downstream tasks, large-scale SSL data is available, or diverse trajectories (e.g., failure cases) can be leveraged to improve world model understanding. However, when data and computational resources are limited, our results show that joint optimization is more effective than the conventional two-stage approach.

G. LIMITATIONS

Despite the significant theoretical implications of our findings, the scope of our empirical evaluation is limited by the benchmarks and available resources. Consistent with

TABLE 3. Comparison of two-stage and joint optimization approaches. Average task success rate (%) across benchmarks. In the two-stage approach, the model is first pretrained with JEPA, after which the encoder is frozen and a newly initialized action decoder is trained using IL. Joint optimization yields higher success rates in all three environments.

Method	Push-T (1 task) ↑	ManiSkill (5 tasks) ↑	Meta-World (15 tasks) ↑
Two-stage approach	27%	0%	23.3%
ACT-JEPA	**41%**	**36%**	**92%**

standard practice in the field, we evaluate the proposed architecture on common simulated benchmarks, which enable controlled and reproducible experimentation. However, the evaluation remains limited in scale and data diversity compared to larger or real-world robotic settings. We summarize the main limitations below.

1) REAL-WORLD ENVIRONMENTS

Without access to real-world robots, we could not deploy and test policies in real-world environments. Such environments present more complex challenges compared to the controlled conditions of simulations.

2) TARGET MODALITY

We trained JEPA with proprioceptive states, leaving modalities such as touch, images, and depth unexplored. While some modalities (e.g., tactile feedback) were unavailable, image data, although highly informative, are high-dimensional and require diverse and larger datasets to yield performance gains.

3) DATASET DIVERSITY

Simulated environments impose inherent limits on dataset diversity. In real-world environments, lighting conditions and objects to manipulate can change significantly, producing a varied and diverse dataset, which is challenging in simulated environments. For example, trajectories collected in Meta-World were recorded under uniform conditions: the same camera view, table, room, and lighting setup. This homogeneity limits the dataset diversity needed to fully harness the advantages of JEPA.

4) DATASET SIZE

Besides the lack of dataset diversity, our study is constrained by the dataset size. Our datasets consist of hundreds of trajectories each, which is small compared to large-scale real-world datasets such as Bridge [50] or Open X-Embodiment [51]. As a self-supervised technique, JEPA is well suited to large and varied datasets. We expect its most significant benefits to emerge as the number of trajectories, task diversity, and available modalities increase. This aligns with trends in vision and language, where large-scale pretraining on broad datasets has led to remarkable success. Scaling ACT-JEPA in this manner could enable it to serve as a foundation policy model. How well the architecture transfers to larger, real-world datasets remains an open question that we plan to explore in future work.

V. CONCLUSION

In this work, we present ACT-JEPA, a novel end-to-end architecture that learns to generate actions using IL, while simultaneously learning a world model using JEPA. By jointly optimizing these objectives, ACT-JEPA learns shared representations that improve both policy performance and world modeling. Our experiments across multiple benchmarks demonstrate that ACT-JEPA consistently outperforms IL baselines in decision-making and world modeling tasks. Additionally, learning a world model with JEPA yields representations that effectively transfer to downstream tasks. While our experiments focus on IL, the learned world model opens opportunities for broader control applications, including planning and alternative prediction objectives. In the future, we aim to scale the architecture with larger datasets and evaluate ACT-JEPA's performance in real-world environments. Overall, these results highlight the advantage of integrating a JEPA-based world model with imitation learning, suggesting a promising direction for more effective and generalizable policies.

REFERENCES

[1] A. Brohan, "RT-1: Robotics transformer for real-world control at scale," in *Proc. Robot., Sci. Syst.*, Daegu, Republic of Korea, Jul. 2023.

[2] A. Brohan et al., "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. Conf. Robot Learn.*, vol. 229, Jul. 2023, pp. 2165–2183.

[3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Robot. Res.*, vol. 44, nos. 10–11, pp. 1684–1704, Sep. 2025.

[4] T. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Proc. 19th Robot., Sci. Syst.*, Jul. 2023, doi: 10.15607/RSS.2023.XIX.016.

[5] Q. Garrido, M. Assran, N. Ballas, A. Bardes, L. Najman, and Y. LeCun, "Learning and leveraging world models in visual representation learning,"2024, *arXiv:2403.00504*.

[6] D. Ha and J. Schmidhuber, "World models," Mar. 2018, *arXiv:1803.10122*.

[7] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Comput. Surv.*, vol. 50, pp. 1–35, Mar. 2018.

[8] P. Wu, A. Majumdar, K. H. Stone, Y. Lin, I. Mordatch, P. Abbeel, and A. Rajeswaran, "Masked trajectory models for prediction, representation, and control," in *Proc. Int. Conf. Mach. Learn.*, vol. 202, May 2023, pp. 37607–37623.

[9] A. Vujinović, N. Luburić, J. Slivka, and A. Kovačević, "Using ChatGPT to annotate a dataset: A case study in intelligent tutoring systems," *Mach. Learn. Appl.*, vol. 16, Jun. 2024, Art. no. 100557.

[10] K. Black et al., "π_0: A vision-language-action flow model for general robot control," 2024, *arXiv:2410.24164*.

[11] Y. LeCun, "A path towards autonomous machine intelligence version 0.9.2, 2022-06-27," Jun. 2022.

[12] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 15619–15629.
[13] A. Bardes et al., "Revisiting feature prediction for learning visual representations from video," *Trans. Mach. Learn. Res.*, Aug. 2024, doi: 10.48550/arxiv.2404.08471. [Online]. Available: https://openreview.net/forum?id=QaCCuDfBk2
[14] H. Lin, T. Nagarajan, N. Ballas, A. Mido, M. Komeili, M. Bansal, and K. Sinha, "VEDIT: Latent prediction architecture for procedural video representation learning," in *Proc. Int. Conf. Learn. Represent.*, Oct. 2024.
[15] S. Yu, S. Kwak, H. Jang, J. Jeong, J. Huang, J. Shin, and S. Xie, "Representation alignment for generation: Training diffusion transformers is easier than you think," in *Proc. Int. Conf. Learn. Represent.*, Dec. 2024.
[16] S. Haldar, Z. Peng, and L. Pinto, "BAKU: An efficient transformer for multi-task policy learning," in *Proc. Adv. Neural Inf. Process. Syst. 37*, Jul. 2024, pp. 141208–141239.
[17] Z. J. Cui, S. Haldar, A. Iyer, H. Pan, and L. Pinto, "DynaMo: In-domain dynamics pretraining for visuo-motor control," in *Proc. Neural Inf. Process. Syst.*, Oct. 2024, pp. 33933–33961.
[18] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "Dino-WM: World models on pre-trained visual features enable zero-shot planning," in *Proc. Int. Conf. Mach. Learn.*, vol. 267, Nov. 2024, pp. 79115–79135.
[19] T. Yu, D. Quillen, Z. He, R. Julian, K. Hausman, C. Finn, and S. Levine, "Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning," 2019, *arXiv:1910.10897*.
[20] S. Tao et al., "Demonstrating GPU parallelized robot simulation and rendering for generalizable embodied AI with ManiSkill3," in *Proc. Robot., Sci. Syst.*, Los Angeles, CA, USA, Jun. 2025, doi: 10.15607/RSS.2025.XXI.021.
[21] R. Bhirangi, V. Pattabiraman, E. Erciyes, Y. Cao, T. Hellebrekers, and L. Pinto, "AnySkin: Plug-and-play skin sensing for robotic touch," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2025, pp. 16563–16570.
[22] M. Lambeta et al., "Digitizing touch with an artificial multimodal fingertip," 2024, *arXiv:2411.02479*.
[23] Q. Zhang, P. Cui, D. Yan, J. Sun, Y. Duan, G. Han, W. Zhao, W. Zhang, Y. Guo, A. Zhang, and R. Xu, "Whole-body humanoid robot locomotion with human reference," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Oct. 2024, pp. 11225–11231.
[24] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "OpenVLA: An open-source vision-language-action model," in *Proc. Conf. Robot Learn.*, vol. 270, Sep. 2024, pp. 2679–2713.
[25] A. A. Altanzaya, Z. Cui, L. Pinto, and N. M. Shafiullah, "Behavior transformers: Cloning K modes with one stone," in *Proc. Adv. Neural Inf. Process. Syst. 35*, Oct. 2022, pp. 22955–22968.
[26] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto, "Behavior generation with latent actions," in *Proc. Int. Conf. Mach. Learn.*, vol. 235, Jun. 2024, pp. 26991–27008.
[27] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "NoMaD: Goal masked diffusion policies for navigation and exploration," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 63–70.
[28] D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. Tan, L. Chen, Q. Vuong, T. Xiao, P. Sanketi, D. Sadigh, C. Finn, and S. Levine, "Octo: An open-source generalist robot policy," in *Proc. 20th Robot., Sci. Syst.*, Jul. 2024, doi: 10.15607/RSS.2024.XX.090.
[29] M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, and I. Sutskever, "Generative pretraining from pixels," in *Proc. Int. Conf. Mach. Learn.*, vol. 1, 2020, pp. 1691–1703.
[30] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
[31] K. He, X. Chen, S. Xie, Y. Li, P. Dollar, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 15979–15988.
[32] Y. Song, Z. Tong, J. Wang, and L. Wang, "VideoMAE: Masked autoencoders are data-efficient learners for self-supervised video pre-training," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 10078–10093.
[33] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 6840–6851.
[34] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, France, Oct. 2023, pp. 4172–4182.
[35] Z. Fei, M. Fan, and J. Huang, "A-JEPA: Joint-embedding predictive architecture can listen," 2024, *arXiv:2311.15830*.
[36] A. Saito, P. Kudeshia, and J. Poovvancheri, "Point-JEPA: A joint embedding predictive architecture for self-supervised learning on point cloud," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Feb. 2025, pp. 7348–7357.
[37] C. Higuera, A. Sharma, C. K. Bodduluri, T. Fan, P. Lancaster, M. Kalakrishnan, M. Kaess, B. Boots, M. Lambeta, T. Wu, and M. Mukadam, "Sparsh: Self-supervised touch representations for vision-based tactile sensing," in *Proc. Conf. Robot Learn.*, vol. 270, 2024, pp. 885–915.
[38] J. Bruce et al., "Genie: Generative interactive environments," in *Proc. 41st Int. Conf. Mach. Learn.*, vol. 235, 2024, pp. 4603–4623.
[39] A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado, "GAIA-1: A generative world model for autonomous driving," 2023, *arXiv:2309.17080*.
[40] J. Urain, A. Mandlekar, Y. Du, N. M. M. Shafiullah, D. Xu, K. Fragkiadaki, G. Chalvatzaki, and J. Peters, "A survey on deep generative models for robot learning from multimodal demonstrations," *IEEE Trans. Robot.*, vol. 42, pp. 60–79, Aug. 2026. [Online]. Available: https://doi.org/10.1109/tro.2025.3631816
[41] M. Yang, Y. Du, K. Ghasemipour, J. Tompson, D. Schuurmans, and P. Abbeel, "Learning interactive real-world simulators," in *Proc. Int. Conf. Learn. Represent.*, Sep. 2023.
[42] A. Bardes, J. Ponce, and Y. LeCun, "VICReg: Variance-invariance-covariance regularization for self-supervised learning," in *Proc. ICLR*, Jan. 2022.
[43] M. Assran et al., "V-JEPA 2: Self-supervised video models enable understanding, prediction and planning," 2025, *arXiv:2506.09985*.
[44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
[45] M. Oquab et al., "DINOv2: Learning robust visual features without supervision," in *Proc. Trans. Mach. Learn. Res.*, Jan. 2024.
[46] E. Perez, F. Strub, H. D. Vries, V. Dumoulin, and A. Courville, "Film: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, Dec. 2018, pp. 3942–3951.
[47] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun, "Navigation world models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2025, pp. 15791–15801.
[48] L. Chen, K. Lü, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 15084–15097.
[49] J. Sturm, W. Burgard, and D. Cremers, "Evaluating egomotion and structure-from-motion approaches using the tum RGB-D benchmark," in *Proc. Workshop Color-Depth Camera Fusion Robot. IEEE/RJS Int. Conf. Intell. Robot Syst. (IROS)*, vol. 13, May 2012, p. 6.
[50] H. Walke, K. Black, A. P. Lee, M. J. Kim, M. Du, C. Zheng, T. Z. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine, "BridgeData V2: A dataset for robot learning at scale," in *Proc. Conf. Robot Learn.*, vol. 229, Jan. 2023, pp. 1723–1736.
[51] A. O'Neill et al., "Open X-embodiment: Robotic learning datasets and RT-X models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2024, pp. 6892–6903, doi: 10.1109/ICRA57147.2024.10611477.

ALEKSANDAR VUJINOVIĆ received the B.Sc. and M.Sc. degrees in computer science from the University of Novi Sad, in 2021 and 2022, respectively. He is currently pursuing the Ph.D. degree. He is a Teaching Assistant with the Faculty of Technical Sciences. His research interests include self-supervised learning, foundational models, world models, and neural network architectures.

ALEKSANDAR KOVAČEVIĆ received the B.Sc. degree in computer science from the University of Novi Sad, in 2003, and the M.Sc. and Ph.D. degrees from the Faculty of Technical Sciences, in 2006 and 2011, respectively. He has been with the Faculty of Technical Sciences, since 2007, becoming a Full Professor, in 2022. He teaches courses in data mining systems, numerical algorithms, and foundations of computational intelligence. He has published more than 80 scientific articles and participated in various research projects. His research interests include text mining and data mining.